CAPÍTULO 4. GIT EN EL SERVIDOR

Introducción

Git es un sistema de control de versiones distribuido que permite a los usuarios trabajar con copias completas de un repositorio. Sin embargo, cuando varias personas participan en un mismo proyecto, es necesario contar con un lugar que permita compartir los cambios y mantener una versión común del trabajo.

Los servidores Git cumplen esta función, ya que permiten almacenar repositorios y facilitar la colaboración entre desarrolladores. Estos servidores pueden estar alojados en plataformas como GitHub o en servidores privados administrados por una organización.

En este capítulo se explican los conceptos fundamentales de los repositorios remotos, la creación de repositorios en un servidor, la conexión con repositorios existentes y los comandos necesarios para enviar y recibir cambios.

1. ¿Qué es un repositorio remoto?

Un repositorio remoto es una versión de un proyecto Git que se encuentra alojada en otro lugar, como un servidor o una plataforma en línea. Su propósito principal es permitir que varias personas compartan el trabajo realizado en un proyecto.

Por ejemplo, una persona puede trabajar en su computador y registrar sus cambios localmente. Posteriormente, puede enviarlos al repositorio remoto para que otros integrantes del equipo tengan acceso a ellos.

Los repositorios remotos también sirven como respaldo del proyecto y facilitan la colaboración desde diferentes computadores.

Es importante tener en cuenta que Git no depende exclusivamente de una conexión a internet para funcionar. Muchas operaciones, como crear confirmaciones o consultar el historial local, pueden realizarse sin conexión. La conexión es necesaria para intercambiar cambios con un servidor remoto.

2. Consultar los repositorios remotos

Para conocer los repositorios remotos asociados a un proyecto, se utiliza el comando git remote.

```
git remote
```

Este comando muestra los nombres de los repositorios remotos configurados. Generalmente, el nombre utilizado para el repositorio principal es origin.

Para consultar los nombres y las direcciones de los repositorios remotos, se utiliza:

```
git remote -v
```

La opción -v permite visualizar las direcciones utilizadas para descargar y enviar cambios.

Por ejemplo, un repositorio remoto puede tener una dirección HTTPS como la siguiente:

https://github.com/usuario/mi-proyecto.git

La dirección anterior es un ejemplo y debe reemplazarse por la dirección real del repositorio que se desea utilizar.

3. **Clonar un repositorio remoto**

Clonar consiste en crear una copia local de un repositorio que se encuentra alojado en un servidor. Esta operación permite descargar el proyecto y comenzar a trabajar en él.

Para clonar un repositorio se utiliza el comando git clone, seguido de su dirección.

```
git clone https://github.com/usuario/mi-proyecto.git
```

Al ejecutar este comando, Git descarga el repositorio y crea una carpeta con los archivos del proyecto, su historial de confirmaciones y la configuración necesaria para conectarse con el repositorio remoto.

Después de clonar el proyecto, se puede ingresar a la carpeta creada:

```
cd mi-proyecto
```

Para comprobar que el repositorio remoto quedó configurado correctamente, se ejecuta:

```
git remote -v
```

Este procedimiento es útil cuando se desea comenzar a trabajar en un proyecto existente o participar en un trabajo colaborativo.

4. **Agregar un repositorio remoto**

Cuando ya existe un repositorio Git en el computador y se desea conectarlo con uno alojado en un servidor, se puede agregar la dirección remota mediante el comando git remote add.

```
git remote add origin https://github.com/usuario/mi-proyecto.git
```

En este ejemplo, origin es el nombre que se asigna al repositorio remoto y la dirección corresponde al repositorio alojado en GitHub.

Después de agregarlo, se recomienda verificar la configuración:

```
git remote -v
```

Este procedimiento permite establecer la conexión entre el proyecto local y el remoto. Si el repositorio ya tiene configurado un remoto con el mismo nombre, será necesario revisar su configuración antes de agregar otro.

5. **Descargar cambios desde un repositorio remoto**

Git ofrece diferentes comandos para recibir información y cambios de un repositorio remoto. Dos de los más importantes son git fetch y git pull.

- **Comando git fetch**

El comando git fetch descarga información actualizada del repositorio remoto, incluyendo nuevas confirmaciones y referencias de ramas, pero no integra automáticamente esos cambios en la rama local actual.

```
git fetch origin
```

Este comando permite consultar los cambios que existen en el servidor antes de decidir cómo incorporarlos al trabajo local.

Es especialmente útil cuando se desea revisar las modificaciones realizadas por otros colaboradores sin alterar inmediatamente la rama en la que se está trabajando.

- **Comando git pull**

El comando git pull permite descargar cambios del repositorio remoto e integrarlos en la rama local actual. Habitualmente, realiza una operación de obtención de cambios y después una integración mediante fusión o rebase, según la configuración utilizada.

```
git pull origin main
```

En este ejemplo, Git obtiene los cambios de la rama main del repositorio remoto llamado origin e intenta integrarlos en la rama local activa.

Si el proyecto utiliza otro nombre para la rama principal, como master, se debe utilizar el nombre correspondiente.

Antes de ejecutar este comando, es recomendable guardar o confirmar los cambios locales para evitar inconvenientes durante la integración.

6. **Enviar cambios al repositorio remoto**

Cuando se realizan modificaciones en el proyecto local y se registran mediante una confirmación, es posible enviarlas al servidor con el comando git push.

```
git push origin main
```

Este comando envía las confirmaciones de la rama local main al repositorio remoto origin, siempre que se cumplan las condiciones necesarias para actualizar la rama.

Si es la primera vez que se envía una rama local y todavía no tiene configurada una rama remota de seguimiento, se puede utilizar:

```
git push -u origin main
```

La opción -u establece la relación de seguimiento entre la rama local y la remota. De esta manera, en operaciones posteriores puede ser suficiente con ejecutar git push o git pull, según la configuración del repositorio.

Es importante recordar que git push envía confirmaciones que ya han sido registradas mediante git commit. Los cambios que todavía no se han confirmado no se envían como parte de una nueva confirmación.

7. **Trabajar con ramas remotas**

Cuando otras personas trabajan en el mismo proyecto, pueden existir ramas en el servidor que todavía no están disponibles como ramas locales de trabajo.

Para consultar las ramas locales y las ramas remotas conocidas por el repositorio, se utiliza:

```
git branch -a
```

Las ramas remotas suelen aparecer con nombres como remotes/origin/nueva-funcionalidad.

Para crear una rama local a partir de una rama remota y comenzar a trabajar en ella, se puede utilizar:

```
git switch -c nueva-funcionalidad --track origin/nueva-funcionalidad
```

Este comando crea la rama local nueva-funcionalidad a partir de la rama remota correspondiente y establece su seguimiento.

Si la rama remota todavía no aparece en el listado, puede ser necesario actualizar la información del servidor mediante:

```
git fetch origin
```

Trabajar con ramas remotas permite que los integrantes de un equipo desarrollen diferentes funcionalidades y compartan sus avances sin necesidad de trabajar directamente sobre una única rama.

8. **Eliminar una rama remota**

Cuando una rama remota deja de ser necesaria, por ejemplo, después de integrar una funcionalidad, puede eliminarse mediante el comando git push con la opción --delete.

```
git push origin --delete nueva-funcionalidad
```

Este comando solicita eliminar la rama nueva-funcionalidad del repositorio remoto origin.

Antes de ejecutarlo, se debe comprobar que la rama ya no sea necesaria y que sus cambios importantes estén integrados. La eliminación de la rama remota no elimina automáticamente las copias locales que puedan existir en los computadores de otros colaboradores.

9. Autenticación y permisos

Para intercambiar cambios con un repositorio remoto, es necesario contar con los permisos correspondientes. Algunas operaciones solo permiten descargar información, mientras que otras, como enviar confirmaciones, requieren autorización de escritura.

En plataformas como GitHub, la autenticación puede realizarse mediante HTTPS o SSH, según la configuración del usuario. En conexiones HTTPS, GitHub no permite utilizar la contraseña habitual de la cuenta como método de autenticación Git; normalmente se utiliza un mecanismo autorizado, como un token personal, el administrador de credenciales o GitHub CLI.

En el caso de SSH, se necesita configurar una clave y asociarla con la cuenta correspondiente.

Si se presenta un error de autenticación o permisos, se recomienda verificar la dirección del repositorio, la cuenta utilizada, las credenciales configuradas y los permisos asignados al proyecto.

Las credenciales y los tokens deben mantenerse privados y nunca deben compartirse públicamente ni incluirse directamente en archivos del proyecto.

10. Ejercicio práctico

Para aplicar los conocimientos de este capítulo, se realizará un ejercicio de conexión entre un repositorio local y uno remoto.

Paso 1. Consultar los repositorios remotos configurados.

```
git remote -v
```

Si no aparece ningún repositorio remoto, se puede agregar uno utilizando la dirección del proyecto correspondiente.

Paso 2. Agregar el repositorio remoto, si es necesario.

git remote add origin https://github.com/usuario/mi-proyecto.git

Se debe reemplazar la dirección de ejemplo por la dirección real del repositorio. Si ya existe un remoto llamado origin, no es necesario ejecutar nuevamente este comando.

Paso 3. Crear un archivo de prueba.

echo "Práctica de repositorios remotos" > remoto.txt

Paso 4. Agregar el archivo al área de preparación.

git add remoto.txt

Paso 5. Registrar el cambio.

git commit -m "Agrega archivo de práctica remota"

Paso 6. Enviar los cambios al repositorio remoto.

Si la rama local se llama main y está vinculada con la rama remota correspondiente, se puede ejecutar:

git push origin main

Si la rama todavía no tiene configurado el seguimiento remoto, se puede utilizar:

git push -u origin main

Este ejercicio requiere que el repositorio remoto exista, que la rama de destino esté correctamente configurada y que el usuario tenga permisos para enviar cambios. Si el repositorio remoto ya contiene archivos o confirmaciones que no están presentes localmente, puede ser necesario sincronizar los historiales antes de realizar el envío.

Conclusión

Los repositorios remotos son fundamentales para compartir proyectos y colaborar con otros usuarios mediante Git. Los comandos git clone, git remote, git fetch, git pull y git push permiten conectar repositorios, obtener actualizaciones y enviar los cambios realizados localmente.

Además, el conocimiento de las ramas remotas, los mecanismos de autenticación y los permisos de acceso contribuye a mantener un flujo de trabajo organizado y seguro. El uso adecuado de estas herramientas facilita el trabajo colaborativo y permite administrar de manera eficiente las diferentes versiones de un proyecto.

Referencias

Chacon, S., y Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. Capítulo 4: Git en el servidor. Disponible en: https://git-scm.com/book/es/v2/Git-en-el-servidor